

PHP / Laravel Developer

Hiring Task — Technical Assessment

Role	PHP / Laravel Developer
Task Duration	3 Days
Stack	Laravel 10+, MySQL, Blade / HTML + JS (Vanilla or Alpine.js)
Difficulty	Mid-level — practical, real-world scenario

Overview

Build a Staff Leave Management System — a small but production-realistic internal web application where employees can apply for leave and managers/admins can manage, approve, or reject those requests.

This task is intentionally modeled after the kind of legacy + Laravel hybrid systems common in real company environments. You are expected to treat this like a genuine work assignment, not a tutorial exercise.

User Roles & Permissions

The system must support three roles with strictly separated permissions.

Employee

- Register and log in.
- Apply for leave (specify dates, type, reason).
- View own leave applications and their current status.
- Cancel a pending leave request.

Manager

- View leave applications of employees assigned to them.
- Approve or reject applications with an optional comment.
- Cannot modify other managers' teams or admin-level data.

Admin

- Full CRUD on employees and managers.
- Assign employees to managers.
- View all leave applications across the organization.
- Manually change leave status (override).
- Manage leave types (e.g., Sick, Casual, Annual).

Functional Requirements

Authentication & Authorization

- Register / Login / Logout using Laravel's built-in auth scaffolding or a manual implementation.
- Role-based access control (RBAC) — routes and views must be guarded by role.
- Middleware to restrict unauthorized access (return 403, not a redirect loop).
- Passwords must be hashed (bcrypt). No plaintext passwords anywhere.

Leave Application Workflow

- Employee selects leave type, start date, end date, and submits a reason.
- System must validate: date range is valid, no overlapping approved leave for the same employee, sufficient notice (start date cannot be in the past).
- Status flow: Pending → Approved / Rejected. Admin can also force-set status.
- Once approved or rejected, the employee cannot edit or cancel the application.
- Manager can add a comment when approving or rejecting.

Employee Dashboard

- List of own leave applications with status badges (Pending, Approved, Rejected).
- Filter by status and leave type.
- Summary card: total leaves taken this year vs. entitlement.

Manager Dashboard

- List of all pending applications from assigned employees.
- Approve / Reject action with optional comment — inline or modal.
- View history of processed applications.

Admin Panel

- CRUD for Users (employees and managers), Leave Types.
- Assign / reassign employees to managers.
- Global view of all applications with filters (status, department, date range).

- Leave entitlement configuration per leave type (e.g., 12 days annual per year).

Technical Requirements

Use Laravel 10+ with MySQL. Frontend should be Blade-based with Vanilla JS or Alpine.js for any interactivity — no heavy JS frameworks. Keep the architecture clean and follow Laravel conventions.

Migrations and seeders are required. The reviewer should be able to clone the repo, run migrations with seeders, and have a fully working system with test credentials for each role.

Bonus (Optional)

These are not required but will strengthen your submission:

- Email notification to employee when leave status changes (use Laravel Mail with Mailtrap or log driver).
- Leave balance auto-calculation (track days used vs. entitlement).
- Export leave report as CSV (Admin only).
- Basic API endpoints (JSON) for the leave application list — useful if a mobile app were added later.
- Write at least 3 feature tests using PHPUnit / Pest covering: login, leave application creation, and role-based access.

Deliverables

1. A GitHub repository (public or shared) with a clear commit history — not a single commit dump.
2. A README.md containing: project setup instructions, .env configuration guide, seeder/migration commands, and login credentials for each role (for the reviewer to test).
3. An .env.example file with all required keys.
4. Database migrations and seeders — the reviewer should be able to run `php artisan migrate --seed` and get a working system.
5. (Optional) A brief note in the README about decisions you made or trade-offs you considered.

Ground Rules

- You may use any Laravel starter kit or scaffolding (Breeze, etc.) for auth setup.
- Third-party packages are allowed but must be listed in `composer.json` — do not vendor-copy packages.

- Do not use a pre-built admin panel (e.g., Filament, Nova, Voyager) — we want to see how you build interfaces yourself.
- AI-generated code is not prohibited, but you should be able to explain every part of your submission in a follow-up call.
- If you get stuck, note it in your README — partial working submissions are evaluated fairly.

Good luck — we look forward to reviewing your work.